

AIML LAB ASSIGNMENT - 2

Name: Siddhesh Abasaheb Mane

PRN: 202401040187

Batch: A2

Roll No.: 46

Title

Comparative Evaluation of Search Algorithms for Autonomous Hospital Delivery Robot Path Planning

Aim

To Evaluate the Performance of Various Algorithms (Uninformed, Informed, Local Search and Constraint Satisfaction) of Problem Solving through Search. The robot must travel from a **Pharmacy (Start)** to an **ICU (Goal)** through a hospital grid while avoiding blocked areas.

Objectives

1. Represent a hospital navigation problem as a state-space search problem.
2. Implement **BFS, DFS and Uniform-Cost Search** as uninformed search algorithms.
3. Implement **Greedy Best-First Search and A*** as informed search algorithms.
4. Implement **Hill Climbing and Genetic Algorithm** as local/evolutionary search methods.
5. Formulate the same navigation problem as a **Constraint Satisfaction Problem (CSP)** and solve it using backtracking.
6. Compare algorithms using path cost, success/failure, search work and execution time.
7. Study the strengths and limitations of each search strategy.

Theory

An AI problem can be represented using:

- **Initial state:** where the agent starts.
- **Goal state:** desired final state.
- **Actions/operators:** possible moves.
- **Transition model:** result of applying an action.
- **Path cost:** cost of the sequence of actions.
- **State space:** all reachable configurations.

Search Categories

Category	Algorithms used
Uninformed Search	BFS, DFS, UCS
Informed Search	Greedy Best-First, A*
Local / Evolutionary Search	Hill Climbing, Genetic Algorithm
Constraint Satisfaction	Backtracking CSP

Common Problem: Autonomous Hospital Delivery Robot

A hospital delivery robot has to transport a medical item from the **Pharmacy** to the **ICU**.

Environment Representation

- **S** = Start / Pharmacy
- **G** = Goal / ICU
- **.** = Free corridor
- **#** = Blocked area

The exact same grid, start state and goal state are used by every algorithm.

State Representation

A state is represented by a coordinate:

(row, column)

Example:

(0, 0) = Pharmacy

(6, 6) = ICU

Actions

The robot can move:

- Up
- Down
- Left
- Right

The robot cannot enter a blocked cell or leave the grid.

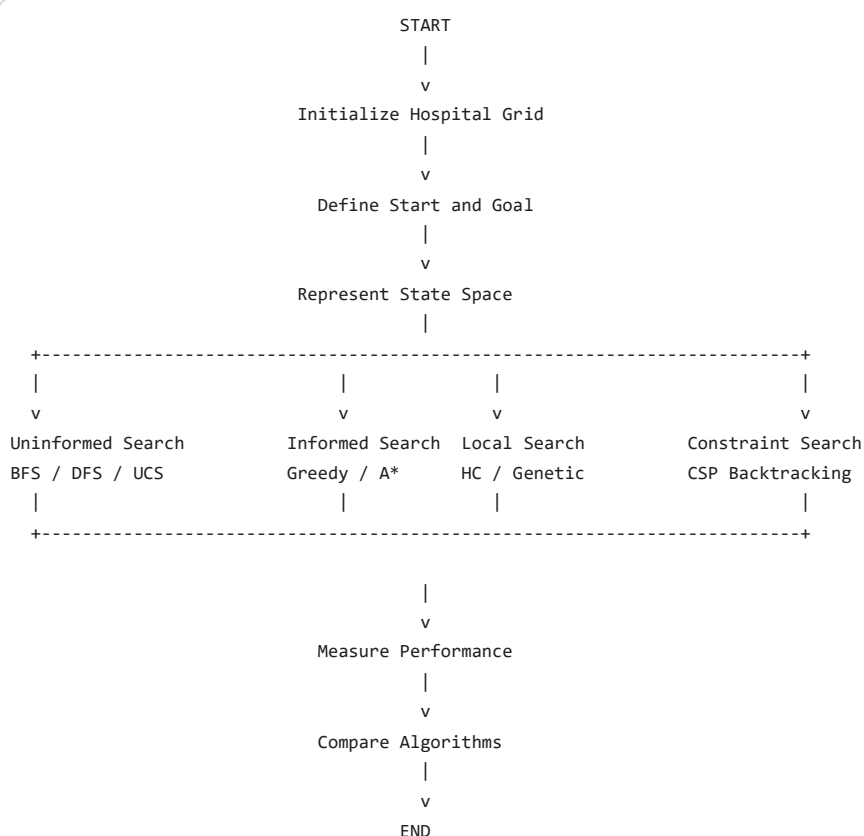
Common Performance Measures

1. **Success:** whether the goal is reached.
2. **Path cost:** number of movements.
3. **States/work evaluated:** algorithm-specific amount of search work.
4. **Execution time:** computational time.
5. **Optimality:** whether the shortest route is obtained.

Common Search Environment

```
S . # . # . .
. . # # . . .
. . . . . # .
. . # # . . .
. # . . . . #
. . . # . . #
. . . # . . #
# # # . # . G
```

Flowchart



Code — Common Environment and Utilities

```

import random
import time
import math
import heapq
from collections import deque

GRID = [
    "S.#.#..",
    "..##...",
    "....#.",
    "..##...",
    ".#...#",
    "...#.#",
    "###.#G"
]

ROWS = len(GRID)
COLS = len(GRID[0])

START = (0, 0)
GOAL = (6, 6)

DIRECTIONS = [
    (1, 0), # Down
    (-1, 0), # Up
    (0, 1), # Right
    (0, -1) # Left
]

def is_valid(position):
    r, c = position
    return (
        0 <= r < ROWS and
        0 <= c < COLS and
        GRID[r][c] != "#"
    )

def neighbors(position):
    r, c = position

    for dr, dc in DIRECTIONS:
        nxt = (r + dr, c + dc)

        if is_valid(nxt):
            yield nxt

def heuristic(position):
    """Manhattan distance to the goal."""
    return abs(position[0] - GOAL[0]) + abs(position[1] - GOAL[1])

def reconstruct_path(parent, goal=GOAL):
    if goal not in parent:
        return None

    path = []
    current = goal

    while current is not None:
        path.append(current)
        current = parent[current]

    return path[::-1]

def path_cost(path):
    if path is None:
        return None
    return len(path) - 1

def show_grid(path=None):
    path_set = set(path or [])

    for r in range(ROWS):
        row = []

        for c in range(COLS):

```

```

        p = (r, c)

        if p == START:
            row.append("S")
        elif p == GOAL:
            row.append("G")
        elif GRID[r][c] == "#":
            row.append("#")
        elif p in path_set:
            row.append("*")
        else:
            row.append(".")

    print(" ".join(row))

print("Hospital Delivery Robot Path Planning")
print("S = Pharmacy / Start")
print("G = ICU / Goal")
print("# = Blocked area")
print(". = Free corridor")
print()
show_grid()

```

```

Hospital Delivery Robot Path Planning
S = Pharmacy / Start
G = ICU / Goal
# = Blocked area
. = Free corridor

S . # . # . .
. . # # . . .
. . . . . # .
. . # # . . .
. # . . . . #
. . . # . . #
# # # . # . G

```

1. Breadth-First Search (BFS)

Idea

BFS uses a **FIFO queue** and explores states level by level.

For unit-cost movements, BFS guarantees the shortest path.

Complexity

- Time: $O(b^d)$
- Space: $O(b^d)$

where b is branching factor and d is the depth of the shallowest solution.

```

# =====
# 1. BREADTH-FIRST SEARCH (BFS) - UNINFORMED SEARCH
# =====

def bfs(start=START, goal=GOAL):
    queue = deque([start])
    parent = {start: None}
    expanded = 0

    while queue:
        current = queue.popleft()
        expanded += 1

        if current == goal:
            break

        for nxt in neighbors(current):
            if nxt not in parent:
                parent[nxt] = current
                queue.append(nxt)

    path = reconstruct_path(parent, goal)
    return path, expanded

bfs_path, bfs_expanded = bfs()

```

```
print("BFS Path:", bfs_path)
print("Path cost:", path_cost(bfs_path))
print("States expanded:", bfs_expanded)
show_grid(bfs_path)
```

```
BFS Path: [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4), (5, 4), (5, 5), (6, 5), (6, 6)]
Path cost: 12
States expanded: 32
S . # . # . .
* . # # . . .
* * * * * # .
. . # # * . .
. # . . * . #
. . . # * * #
# # # . # * G
```

2. Depth-First Search (DFS)

Idea

DFS uses a **LIFO stack** and explores one branch deeply before backtracking.

DFS may find a solution quickly, but it does **not guarantee the shortest path**.

Complexity

- Time: $O(b^m)$
- Space: $O(bm)$

where m is the maximum search depth.

```
# =====
# 2. DEPTH-FIRST SEARCH (DFS) - UNINFORMED SEARCH
# =====

def dfs(start=START, goal=GOAL):
    stack = [start]
    parent = {start: None}
    expanded = 0

    while stack:
        current = stack.pop()
        expanded += 1

        if current == goal:
            break

        next_states = list(neighbors(current))

        for nxt in reversed(next_states):
            if nxt not in parent:
                parent[nxt] = current
                stack.append(nxt)

        path = reconstruct_path(parent, goal)
        return path, expanded

dfs_path, dfs_expanded = dfs()

print("DFS Path:", dfs_path)
print("Path cost:", path_cost(dfs_path))
print("States expanded:", dfs_expanded)
show_grid(dfs_path)
```

```
DFS Path: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (5, 0), (5, 1), (5, 2), (4, 2), (4, 3), (4, 4), (5, 4), (5, 5), (6, 5), (6, 6)]
Path cost: 14
States expanded: 15
S . # . # . .
* . # # . . .
* . . . . # .
* . # # . . .
* # * * . #
* * * # * * #
# # # . # * G
```

3. Uniform-Cost Search (UCS)

Idea

UCS always expands the state with the lowest accumulated path cost $g(n)$.

Here every movement costs 1, so UCS and BFS have equivalent optimality behavior.

Complexity

Depends on path costs; for unit-cost edges it behaves similarly to BFS.

```
# =====
# 3. UNIFORM-COST SEARCH (UCS) - UNINFORMED SEARCH
# =====
# Every movement has cost 1.

def uniform_cost_search(start=START, goal=GOAL):
    priority_queue = [(0, start)]
    parent = {start: None}
    best_cost = {start: 0}
    expanded = 0

    while priority_queue:
        cost, current = heapq.heappop(priority_queue)

        if cost != best_cost[current]:
            continue

        expanded += 1

        if current == goal:
            break

        for nxt in neighbors(current):
            new_cost = cost + 1

            if new_cost < best_cost.get(nxt, math.inf):
                best_cost[nxt] = new_cost
                parent[nxt] = current
                heapq.heappush(priority_queue, (new_cost, nxt))

    path = reconstruct_path(parent, goal)
    return path, expanded

ucs_path, ucs_expanded = uniform_cost_search()

print("UCS Path:", ucs_path)
print("Path cost:", path_cost(ucs_path))
print("States expanded:", ucs_expanded)
show_grid(ucs_path)

UCS Path: [(0, 0), (0, 1), (1, 1), (2, 1), (2, 2), (2, 3), (2, 4), (3, 4), (3, 5), (4, 5), (5, 5), (6, 5), (6, 6)]
Path cost: 12
States expanded: 32
S * # . # . .
. * # # . . .
. * * * # .
. . # # * * .
. # . . * #
. . # . * #
# # # . # * G
```

4. Greedy Best-First Search

Idea

Greedy Best-First Search uses only the heuristic:

$$f(n) = h(n)$$

Here:

$$h(n) = \text{Manhattan distance to the ICU}$$

It chooses the state that appears closest to the goal.

It can be fast, but it is **not guaranteed to find an optimal path**.

```
# =====
# 4. GREEDY BEST-FIRST SEARCH - INFORMED SEARCH
# =====

def greedy_best_first(start=START, goal=GOAL):
    priority_queue = [(heuristic(start), start)]
    parent = {start: None}
    visited = set()
    expanded = 0

    while priority_queue:
        _, current = heapq.heappop(priority_queue)

        if current in visited:
            continue

        visited.add(current)
        expanded += 1

        if current == goal:
            break

        for nxt in neighbors(current):
            if nxt not in visited:
                parent[nxt] = current
                heapq.heappush(
                    priority_queue,
                    (heuristic(nxt), nxt)
                )

    path = reconstruct_path(parent, goal)
    return path, expanded

greedy_path, greedy_expanded = greedy_best_first()

print("Greedy Path:", greedy_path)
print("Path cost:", path_cost(greedy_path))
print("States expanded:", greedy_expanded)
show_grid(greedy_path)
```

```
Greedy Path: [(0, 0), (0, 1), (1, 1), (2, 1), (2, 2), (2, 3), (2, 4), (3, 4), (3, 5), (4, 5), (5, 5), (6, 5), (6, 6)]
Path cost: 12
States expanded: 14
S * # . # . .
. * # . . .
. * * * # .
. . # * * .
. # . . * #
. . # . * #
# # . # * G
```

5. A* Search

Idea

A* combines actual cost and estimated remaining cost:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ = cost from Pharmacy to the current cell.
- $h(n)$ = Manhattan distance from current cell to ICU.

For this unit-cost four-direction grid, Manhattan distance is an admissible heuristic, so A* can find an optimal path.

```
# =====
# 5. A* SEARCH - INFORMED SEARCH
# =====

def a_star(start=START, goal=GOAL):
    priority_queue = [(heuristic(start), 0, start)]
```

```

parent = {start: None}
best_cost = {start: 0}
expanded = 0

while priority_queue:
    f, g, current = heapq.heappop(priority_queue)

    if g != best_cost[current]:
        continue

    expanded += 1

    if current == goal:
        break

    for nxt in neighbors(current):
        new_g = g + 1

        if new_g < best_cost.get(nxt, math.inf):
            best_cost[nxt] = new_g
            parent[nxt] = current

            new_f = new_g + heuristic(nxt)

            heapq.heappush(
                priority_queue,
                (new_f, new_g, nxt)
            )

    path = reconstruct_path(parent, goal)
    return path, expanded

astar_path, astar_expanded = a_star()

print("A* Path:", astar_path)
print("Path cost:", path_cost(astar_path))
print("States expanded:", astar_expanded)
show_grid(astar_path)

```

```

A* Path: [(0, 0), (0, 1), (1, 1), (2, 1), (2, 2), (2, 3), (2, 4), (3, 4), (3, 5), (4, 5), (5, 5), (6, 5), (6, 6)]
Path cost: 12
States expanded: 24
S * # . # . .
. * # # . . .
. * * * * # .
. . # # * * .
. # . . . * #
. . . # . * #
# # # . # * G

```

6. Hill Climbing

Idea

Hill Climbing keeps only the current state and moves to a better neighboring state.

Here lower Manhattan distance is treated as better.

Limitation Demonstrated

The grid contains a situation where the robot can choose an equally attractive local direction that eventually reaches a state from which every immediate move looks worse. Hill Climbing then stops at a **local minimum**, even though a valid goal path exists elsewhere.

```

# =====
# 6. HILL CLIMBING - LOCAL SEARCH
# =====

def hill_climbing(start=START, goal=GOAL):
    current = start
    path = [current]
    states_evaluated = 0

    while current != goal:
        candidates = list(neighbors(current))
        states_evaluated += len(candidates)

```



```

        if not candidates:
            return None, path, states_evaluated

        best = min(candidates, key=heuristic)

        # No improvement -> local minimum / plateau.
        if heuristic(best) >= heuristic(current):
            return None, path, states_evaluated

        current = best
        path.append(current)

    return path, path, states_evaluated

hill_result, hill_history, hill_evaluated = hill_climbing()

print("Hill Climbing success:", hill_result is not None)
print("Path/history:", hill_history)
print("States evaluated:", hill_evaluated)

if hill_result:
    print("Path cost:", path_cost(hill_result))
else:
    print("Hill Climbing stopped at a local minimum.")
show_grid(hill_history)

```

```

Hill Climbing success: False
Path/history: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (5, 0), (5, 1), (5, 2)]
States evaluated: 19
Hill Climbing stopped at a local minimum.
S . # . # . .
* . # # . . .
* . . . . # .
* . # # . . .
* # . . . . #
* * * # . . #
# # # . # . G

```

7. Genetic Algorithm

Representation

A chromosome is a sequence of movement directions:

- 0 = Down
- 1 = Up
- 2 = Right
- 3 = Left

Process

```

Initial Population
    |
    v
Fitness Evaluation
    |
    v
Selection
    |
    v
Crossover
    |
    v
Mutation
    |
    v
New Population
    |
+-----> Repeat

```

Fitness

A successful route receives a large reward. Shorter successful routes receive higher fitness, while invalid movements are penalized.

Because the algorithm is stochastic, it is not guaranteed to produce the shortest route.

```
# =====
# 7. GENETIC ALGORITHM - LOCAL / EVOLUTIONARY SEARCH
# =====

def simulate_chromosome(chromosome):
    position = START
    path = [position]
    invalid_moves = 0

    for gene in chromosome:
        dr, dc = DIRECTIONS[gene]
        nxt = (position[0] + dr, position[1] + dc)

        if is_valid(nxt):
            position = nxt
            path.append(position)

            if position == GOAL:
                return position, path, invalid_moves
            else:
                invalid_moves += 1

    return position, path, invalid_moves

def genetic_algorithm(
    population_size=200,
    generations=500,
    chromosome_length=20,
    mutation_rate=0.05,
    seed=42
):
    rng = random.Random(seed)

    def fitness(chromosome):
        position, path, invalid = simulate_chromosome(chromosome)

        if position == GOAL:
            steps = path.index(GOAL)
            return 10000 - 10 * steps - 5 * invalid

        # For unfinished routes, reward closeness to the goal.
        return (
            -100 * heuristic(position)
            - 5 * invalid
            + len(set(path))
        )

    population = [
        [rng.randrange(4) for _ in range(chromosome_length)]
        for _ in range(population_size)
    ]

    best_chromosome = None
    best_fitness = -float("inf")
    evaluations = 0

    for generation in range(generations):
        scored = [
            (fitness(chromosome), chromosome)
            for chromosome in population
        ]
        evaluations += len(population)

        scored.sort(
            key=lambda item: item[0],
            reverse=True
        )

        if scored[0][0] > best_fitness:
            best_fitness = scored[0][0]
            best_chromosome = scored[0][1][:]

        best_position, best_path, _ = simulate_chromosome(
            best_chromosome
        )

        if best_position == GOAL:
```

```

        return (
            best_path,
            evaluations,
            generation + 1,
            best_fitness
        )

    # Elitism
    new_population = [
        chromosome[:]
        for _, chromosome in scored[:10]
    ]

    def select_parent():
        tournament = rng.sample(scored, 3)
        return max(tournament, key=lambda x: x[0])[1]

    while len(new_population) < population_size:
        parent1 = select_parent()
        parent2 = select_parent()

        cut = rng.randint(1, chromosome_length - 1)

        child = (
            parent1[:cut] +
            parent2[cut:]
        )

        # Mutation
        for i in range(chromosome_length):
            if rng.random() < mutation_rate:
                child[i] = rng.randrange(4)

        new_population.append(child)

    population = new_population

    position, path, _ = simulate_chromosome(best_chromosome)

    if position == GOAL:
        return path, evaluations, generations, best_fitness

    return None, evaluations, generations, best_fitness

ga_path, ga_evaluations, ga_generations, ga_fitness = (
    genetic_algorithm(seed=42)
)

print("Genetic Algorithm success:", ga_path is not None)
print("Path cost:", path_cost(ga_path))
print("Fitness evaluations:", ga_evaluations)
print("Generations:", ga_generations)
print("Best fitness:", ga_fitness)

if ga_path:
    show_grid(ga_path)

```

```

Genetic Algorithm success: True
Path cost: 14
Fitness evaluations: 800
Generations: 4
Best fitness: 9845
S . # . # . .
* * # # . . .
. * * * * # .
. . # # * . .
. # . . * . #
. . . # * * #
# # # . # * G

```

8. Constraint Satisfaction Problem (CSP)

The same pathfinding problem is expressed as a CSP.

Variables

For a path of moves:

Each variable represents the robot's cell at that time step.

Domains

All non-blocked cells of the hospital grid.

Constraints

1. X_0 = Pharmacy
2. X_K = ICU
3. X_i and X_{i+1} must be adjacent.
4. **AllDifferent**: the robot does not repeat a cell.

The solver starts from the Manhattan lower bound and increases the allowed path length until it finds a feasible assignment. Therefore, the first solution is a shortest simple path.

```
# =====
# 8. CONSTRAINT SATISFACTION PROBLEM (CSP)
# =====

def csp_fixed_length(k, start=START, goal=GOAL):
    assignment = {0: start}
    candidate_checks = 0
    backtracks = 0

    def solve(index):
        nonlocal candidate_checks, backtracks

        # Last variable must be the goal.
        if index == k:
            if goal in neighbors(assignment[index - 1]):
                if goal not in assignment.values():
                    candidate_checks += 1
                    assignment[index] = goal
                    return True
            return False

        candidates = [
            cell
            for cell in neighbors(assignment[index - 1])
            if cell not in assignment.values()
        ]

        candidate_checks += len(candidates)

        for cell in candidates:
            assignment[index] = cell

            if solve(index + 1):
                return True

            del assignment[index]
            backtracks += 1

        return False

    if k == 0:
        return ([start] if start == goal else None, 0, 0)

    if solve(1):
        return (
            [assignment[i] for i in range(k + 1)],
            candidate_checks,
            backtracks
        )

    return None, candidate_checks, backtracks

def csp_shortest_path(start=START, goal=GOAL):
    lower_bound = heuristic(start)
    max_length = ROWS * COLS - 1

    total_checks = 0
    total_backtracks = 0

    for k in range(lower_bound, max_length + 1):
        path, checks, backtracks = csp_fixed_length(
```

```

        k, start, goal
    )

    total_checks += checks
    total_backtracks += backtracks

    if path is not None:
        return (
            path,
            total_checks,
            total_backtracks,
            k
        )

    return None, total_checks, total_backtracks, None

csp_path, csp_checks, csp_backtracks, csp_length = (
    csp_shortest_path()
)

print("CSP success:", csp_path is not None)
print("Path cost:", path_cost(csp_path))
print("Candidate checks:", csp_checks)
print("Backtracks:", csp_backtracks)
show_grid(csp_path)

```

```

CSP success: True
Path cost: 12
Candidate checks: 59
Backtracks: 40
S . # . # . .
* . # # . . .
* * * * # .
. . # # * . .
. # . . * . #
. . . # * * #
# # # . # * G

```

9. Performance Comparison

The following cell runs every algorithm on the **same problem instance**.

Important interpretation

The work measures are not identical internally:

- Graph search → number of states expanded.
- Hill Climbing → states/neighbors evaluated.
- Genetic Algorithm → fitness evaluations.
- CSP → candidate assignments checked.

Therefore, the most meaningful cross-algorithm comparisons are **success, path cost, and the qualitative behavior**, while the work/time columns are useful within the context of each algorithm.

```

# =====
# 9. PERFORMANCE COMPARISON
# =====

def timed_call(function, *args, **kwargs):
    start_time = time.perf_counter()
    result = function(*args, **kwargs)
    elapsed_ms = (time.perf_counter() - start_time) * 1000
    return result, elapsed_ms

results = []

# Uninformed
(result, ms) = timed_call(bfs)
path, expanded = result
results.append({
    "Algorithm": "BFS",
    "Category": "Uninformed",
    "Success": path is not None,
    "Path Cost": path_cost(path),
    "Work Measure": expanded,

```

```

        "Work Type": "States expanded",
        "Time (ms)": round(ms, 4)
    })

    (result, ms) = timed_call(dfs)
    path, expanded = result
    results.append({
        "Algorithm": "DFS",
        "Category": "Uninformed",
        "Success": path is not None,
        "Path Cost": path_cost(path),
        "Work Measure": expanded,
        "Work Type": "States expanded",
        "Time (ms)": round(ms, 4)
    })

    (result, ms) = timed_call(uniform_cost_search)
    path, expanded = result
    results.append({
        "Algorithm": "UCS",
        "Category": "Uninformed",
        "Success": path is not None,
        "Path Cost": path_cost(path),
        "Work Measure": expanded,
        "Work Type": "States expanded",
        "Time (ms)": round(ms, 4)
    })

    # Informed
    (result, ms) = timed_call(greedy_best_first)
    path, expanded = result
    results.append({
        "Algorithm": "Greedy Best-First",
        "Category": "Informed",
        "Success": path is not None,
        "Path Cost": path_cost(path),
        "Work Measure": expanded,
        "Work Type": "States expanded",
        "Time (ms)": round(ms, 4)
    })

    (result, ms) = timed_call(a_star)
    path, expanded = result
    results.append({
        "Algorithm": "A*",
        "Category": "Informed",
        "Success": path is not None,
        "Path Cost": path_cost(path),
        "Work Measure": expanded,
        "Work Type": "States expanded",
        "Time (ms)": round(ms, 4)
    })

    # Local Search
    (result, ms) = timed_call(hill_climbing)
    path, history, evaluated = result
    results.append({
        "Algorithm": "Hill Climbing",
        "Category": "Local",
        "Success": path is not None,
        "Path Cost": path_cost(path),
        "Work Measure": evaluated,
        "Work Type": "Neighbor states evaluated",
        "Time (ms)": round(ms, 4)
    })

    (result, ms) = timed_call(genetic_algorithm, seed=42)
    path, evaluations, generations, fitness = result
    results.append({
        "Algorithm": "Genetic Algorithm",
        "Category": "Local / Evolutionary",
        "Success": path is not None,
        "Path Cost": path_cost(path),
        "Work Measure": evaluations,
        "Work Type": "Fitness evaluations",
        "Time (ms)": round(ms, 4)
    })

    # CSP
    (result, ms) = timed_call(csp_shortest_path)
    path, checks, backtracks, length = result

```

```

results.append({
    "Algorithm": "CSP Backtracking",
    "Category": "Constraint Satisfaction",
    "Success": path is not None,
    "Path Cost": path_cost(path),
    "Work Measure": checks,
    "Work Type": "Candidate assignments checked",
    "Time (ms)": round(ms, 4)
})

comparison = pd.DataFrame(results)

display(comparison)

print("\nImportant interpretation:")
print("- BFS, UCS and A* find a 12-move shortest path on this unit-cost grid.")
print("- DFS finds a valid path but it is not shortest.")
print("- Greedy uses only the heuristic and expands fewer states here, but it is not generally guaranteed optimal.")
print("- Hill Climbing gets trapped because a locally worse move is required to escape its chosen route.")
print("- Genetic Algorithm finds a feasible route, but its route length is stochastic and usually not optimal.")
print("- CSP backtracking finds a shortest feasible path when path length is increased from the lower bound.")

```

	Algorithm	Category	Success	Path Cost	Work Measure	Work Type	Time (ms)	
0	BFS	Uninformed	True	12.0	32	States expanded	0.0807	
1	DFS	Uninformed	True	14.0	15	States expanded	0.0647	
2	UCS	Uninformed	True	12.0	32	States expanded	0.0984	
3	Greedy Best-First	Informed	True	12.0	14	States expanded	0.0703	
4	A*	Informed	True	12.0	24	States expanded	0.0774	
5	Hill Climbing	Local	False	NaN	19	Neighbor states evaluated	0.0311	
6	Genetic Algorithm	Local / Evolutionary	True	14.0	800	Fitness evaluations	16.6408	
7	CSP Backtracking	Constraint Satisfaction	True	12.0	59	Candidate assignments checked	0.1699	

Important interpretation:

- BFS, UCS and A* find a 12-move shortest path on this unit-cost grid.
- DFS finds a valid path but it is not shortest.
- Greedy uses only the heuristic and expands fewer states here, but it is not generally guaranteed optimal.
- Hill Climbing gets trapped because a locally worse move is required to escape its chosen route.
- Genetic Algorithm finds a feasible route, but its route length is stochastic and usually not optimal.
- CSP backtracking finds a shortest feasible path when path length is increased from the lower bound.

Next steps: [Generate code with comparison](#) [New interactive sheet](#)

Result Analysis

The algorithms can be evaluated using the following observations:

Uninformed Search

- **BFS** finds a shortest 12-move route because every movement has equal cost.
- **DFS** reaches the ICU but can return a longer route because it commits to a branch before exploring alternatives.
- **UCS** also finds a shortest route. Since every movement costs 1, UCS is effectively equivalent to BFS in this experiment.

Informed Search

- **Greedy Best-First Search** uses the Manhattan heuristic and usually explores fewer states in this grid.
- It is not generally guaranteed to produce an optimal route.
- **A*** combines actual path cost and heuristic information and obtains a shortest route in this unit-cost grid.

Local Search

- **Hill Climbing** uses only local improvement and gets stuck in a local minimum on this particular grid.
- **Genetic Algorithm** searches using a population of candidate action sequences. It can find a feasible route, but the route may contain unnecessary movements and is not guaranteed to be optimal.

Constraint Satisfaction

- **CSP Backtracking** represents the route as variables with domains and constraints.
- By testing increasing path lengths, it finds a shortest feasible path for this problem.
- The main cost is the number of assignments/backtracks explored during constraint search.

Expected Comparison from the Fixed Experiment

A typical run with the provided fixed grid and random seed produces approximately:

Algorithm	Category	Success	Path Cost
BFS	Uninformed	Yes	12
DFS	Uninformed	Yes	14
UCS	Uninformed	Yes	12
Greedy Best-First	Informed	Yes	12
A*	Informed	Yes	12
Hill Climbing	Local	No	Gets stuck
Genetic Algorithm	Local / Evolutionary	Yes	14
CSP Backtracking	Constraint Satisfaction	Yes	12

Exact execution time and stochastic local-search results can vary slightly between runs and hardware. The Genetic Algorithm use a fixed random seed for reproducibility.

Experiment Outcome

The experiment successfully implements four major approaches to problem solving through search using one common hospital robot path-planning problem:

1. **Uninformed:** BFS, DFS, UCS
2. **Informed:** Greedy Best-First Search, A*
3. **Local / Evolutionary:** Hill Climbing, Genetic Algorithm
4. **Constraint Satisfaction:** Backtracking CSP

Using the same environment makes the differences between algorithms easier to observe.

Conclusion

The experiment demonstrates that different search strategies behave differently even when solving the same problem.

BFS and UCS provide shortest routes under the unit-cost assumption, while **DFS** may return a longer route. **Greedy Best-First Search** uses heuristic information to focus the search, while **A*** combines path cost and heuristic information to obtain an optimal route for this grid.

Local-search methods require much less explicit search-tree memory but can face local minima and stochastic behavior. **Hill Climbing** demonstrates the local-minimum problem, while **Genetic Algorithm** provide mechanisms for broader exploration.

The **CSP formulation** shows that the same path-planning problem can also be represented using variables, domains, and constraints. This experiment therefore provides a practical comparison of multiple AI problem-solving strategies on a common example.

Colab Link

[click here](#)

Github Link

[click here](#)